

บทที่ 2

ภาษาโปรแกรม

2.1 ภาษาโปรแกรม

ภาษาคอมพิวเตอร์ที่พัฒนากันมาจากรากอดีตจนถึงปัจจุบันนี้ส่วนใหญ่จะเป็นภาษาที่มีการกำหนดลำดับขั้นตอนในการทำงานที่แน่นอน ซึ่งเราเรียกกันว่าเป็น Procedural Programming Language ตัวอย่างเช่น ภาษาเบสิก ภาษาซี เป็นต้น ในการใช้งานภาษาเหล่านี้ ผู้ที่จะเขียนโปรแกรมจะต้องกำหนดลำดับขั้นตอน ซึ่งเรียกว่า Procedure หรือ Algorithm สำหรับการแก้ปัญหาขึ้นเสียก่อน จากนั้นจึงเริ่มเขียนโปรแกรมให้สามารถทำงานตามลำดับขั้นตอนที่เราได้กำหนดไว้แล้วอย่างแน่นอน โดยไม่มีความยืดหยุ่นในการปรับเปลี่ยนลำดับขั้นตอนได้ ดังจะเห็นได้จากการเขียน Flow Chart ของทุกโปรแกรม จะแสดงถึงลำดับการ Execute ที่แน่นอน อย่างไรก็ตามยังมีกลุ่มของปัญหาอีกกลุ่มหนึ่ง ซึ่งไม่สามารถที่จะใช้ภาษาคอมพิวเตอร์ประเภท Procedural Programming ในการช่วยแก้ปัญหาได้ หรือกระทำได้อย่างยากมาก และไม่มีประสิทธิภาพ กลุ่มของปัญหานี้เรียกชื่อกันว่า Artificial Intelligence (AI) หรืออาจจะอธิบายเป็นภาษาธรรมดาได้ว่าเป็นสิ่งประดิษฐ์ที่ชาญฉลาด ดังนั้นในช่วงปี 1970 จึงได้มีการพัฒนาภาษาเพื่อใช้กับ AI ขึ้น 2 ภาษาคือ ภาษา LISP และภาษาโปรแกรมซึ่งนั้นเป็นคำย่อมาจากคำว่า Programming in Logic หรือการเขียนโปรแกรมโดยวิธีทางตรรกวิทยา

ภาษาโปรแกรมมีลักษณะตรงกันข้ามกับพวก Procedural Programming ที่เด่นชัดก็คือ ความสามารถในการวินิจฉัย และสรุปความเห็นโดยการชักเหตุผลจากข้อเท็จจริง โดยอาศัยฐานความรู้ (Knowledge Base) ที่มีอยู่ในโปรแกรมประกอบกับกลไกการวินิจฉัย (Inference Engine) ที่เราจะสร้างให้กับโปรแกรม โดยอาศัยการทำงานทางด้านตรรกวิทยา ดังนั้นภาษาโปรแกรมจึงเหมาะสำหรับการนำไปแก้ปัญหาที่มีโครงสร้างไม่แน่นอน ซึ่งโปรแกรมในภาษาที่ใช้อยู่ทั่วไปไม่สามารถกระทำได้ หรือสามารถทำได้แต่ไม่มีประสิทธิภาพ ในทางกลับกันภาษาโปรแกรมก็ไม่เหมาะที่จะนำไปใช้กับปัญหาที่มีการคำนวณหรือทำงานตามลำดับขั้นตอนที่แน่ชัดแล้ว อย่างไรก็ตามการพัฒนาภาษาโปรแกรมจนถึงปัจจุบันนี้ก็ยังไม่บรรลุถึงวัตถุประสงค์สูงสุดของการนำไปใช้กับ AI ซึ่งเราจะเห็นได้ว่าภาษายังมีลักษณะของ Procedural Programming อยู่บ้าง ตัวอย่างความเหมาะสมของการใช้ภาษาโปรแกรมใน AI คือ

- ระบบผู้เชี่ยวชาญ (Expert System)
- การโต้ตอบด้วยภาษาธรรมชาติ (Natural Language Processing)
- การสร้างเกมที่ต้องใช้การชักงูด้วยเหตุผล
- การประยุกต์ใช้กับหุ่นยนต์อุตสาหกรรม (Robotics)

2.2 หลักการขั้นต้นของภาษาโปรแกรม

ภาษาโปรแกรม เป็นภาษาคอมพิวเตอร์ที่ใช้สำหรับการแก้ปัญหาซึ่งเกี่ยวข้องกับวัตถุ (Objects) และความสัมพันธ์ระหว่างวัตถุ (Relation) หลักการสำคัญขั้นต้นสำหรับโปรแกรมก็คือ การทำความเข้าใจกับคำว่า Facts, Rules และ Variables ส่วนใหญ่ของโปรแกรมโปรแกรมประกอบด้วยรวบรวมฐานความรู้เกี่ยวกับเรื่องใดเรื่องหนึ่งโดยเฉพาะ ฐานความรู้นี้จะถูกเรียบเรียงอยู่ในรูปแบบของข้อเท็จจริง (Facts) และกฎ (Rules) สมมติว่าเรา

ต้องการจะบอกโปรแกรมเกี่ยวกับข้อเท็จจริงว่า เมธีชอบอรุณญา หรือ Mathee likes Arunya เราจะสามารถเขียนเป็นรูปแบบมาตรฐานในโปรแกรมดังนี้คือ

likes(mathee, arunya).

ในที่นี้ทั้ง mathee และ arunya เรียกว่าวัตถุ (Objects) ส่วน likes เรียกว่าความสัมพันธ์ (Relation) นั่นคือโปรแกรมจะบอกถึงความสัมพันธ์ระหว่างวัตถุคือเมธี และอรุณญา ว่าเมธีชอบอรุณญา สิ่งสำคัญที่เป็นข้อสังเกตคือ

1. ทั้งชื่อของวัตถุและความสัมพันธ์จะต้องขึ้นต้นด้วยตัวอักษรตัวเล็ก
2. ในการเขียนประโยค ให้เขียนความสัมพันธ์ก่อน ส่วนวัตถุให้เขียนอยู่ในวงเล็บ และแยกออกจากกันด้วยเครื่องหมายจุลภาค (,)
3. เมื่อสิ้นสุดประโยคของข้อเท็จจริงจะต้องมีเครื่องหมายหัพภาค (.) เสมอ

ในขณะที่เราให้ค่านิยามแสดงความสัมพันธ์ระหว่างวัตถุที่อยู่ภายในวงเล็บ เราจะต้องให้ความสำคัญของการลำดับการเอ่ยถึงวัตถุ เช่น

likes(mathee, arunya). หมายถึง “mathee likes arunya.”

likes(arunya, mathee). หมายถึง “arunya likes mathee.”

ซึ่งความหมายของทั้งสองประโยคข้างบนนี้จะไม่เหมือนกัน ให้เราลองมาพิจารณาการเขียนข้อเท็จจริงในโปรแกรมอีกดังนี้

valuable(diamond). หมายถึง “diamond is valuable.”

owns(mathee, diamond). หมายถึง “mathee owns diamond.”

father(mathee, suvit). หมายถึง “mathee is suvit’s father.”

ซึ่งเราจะเห็นได้ว่า การตีความหมายของประโยคขึ้นอยู่กับทำให้ความหมายของผู้เขียนโปรแกรมเอง ดังจะเห็นได้ว่า เราอาจจะตีความหมายในประโยคสุดท้ายว่า “mathee’s father is suvit.” ก็ได้ ฉะนั้นในการเขียนแต่ละประโยคในโปรแกรม ผู้เขียนจะต้องให้ความหมายอย่างสมนัยกันตลอด โปรแกรม วัตถุ หรือ object ที่อยู่ภายในวงเล็บมีชื่อเรียกอีกอย่างหนึ่งว่า Argument และความสัมพันธ์ หรือ Relation ที่อยู่ข้างหน้าวงเล็บมีชื่อเรียกอีกอย่างหนึ่งว่า Functor ในแต่ละประโยคเราจะให้มีวัตถุจำนวนเท่าใดก็ได้ เช่น

watch(mathee, arunya, movie). หมายถึง “mathee and arunya watch movie.”

play(mathee, thanit, tennis). หมายถึง “mathee and thanit play tennis.”

ประโยคทุกประโยคที่แสดงเป็นตัวอย่างมานี้เราเรียกว่า ข้อเท็จจริง (Facts) เมื่อเรามีข้อเท็จจริงเก็บสะสมอยู่ในโปรแกรมแล้วก็เท่ากับว่าเราได้จัดให้โปรแกรมมีความรู้ระดับหนึ่งเช่นกัน ฉะนั้นเราจึงสามารถที่จะสอบถามความรู้จากโปรแกรมได้ การสอบถามหรือตั้งคำถามใน Prolog นั้น ก็คือการตั้งเป้าประสงค์ (Goal) นั่นเอง สมมติว่าเรามีข้อเท็จจริงในโปรแกรมดังกล่าวมาแล้ว เมื่อเราตั้งคำถามเราจะได้รับการตอบโต้ลักษณะดังนี้คือ

คำถาม Goal : valuable (diamond).

คำตอบ True

คำถาม Goal : likes(mathee,arunya).

คำตอบ True

คำถาม Goal : father(mathee, chaiyan).

คำตอบ False

ในที่นี้โปรล็อกจะให้คำตอบว่า “เป็นจริง” หรือ “เป็นเท็จ” โดยอาศัยข้อเท็จจริงที่มีอยู่ในโปรแกรมแล้ว การที่โปรล็อกให้คำตอบได้ถูกต้องก็เพราะว่า เมื่อเราตั้งคำถามหรือเป่าประสงค์แล้วโปรล็อกจะทำการค้นหาข้อเท็จจริงที่ตรงกับคำถามของเรา ถ้าโปรล็อกค้นพบก็จะตอบว่า True แต่ถ้าหาไม่พบก็จะตอบว่า False ให้สังเกตว่าคำถามและข้อเท็จจริงจะมีประโยคและตัวอักษรเหมือนกันทุกประการ

การตั้งคำถามเหมือนข้างบนนี้ยังไม่เป็นการคล่องตัวพอสำหรับการใช้งานโปรล็อก ในทางปฏิบัติ เราควรจะถามดังนี้

คำถาม Goal : likes(mathee, Who).

คำตอบ Who = arunya

คำถาม Goal : father(Father, Son).

คำตอบ Father = mathee

Son = suvit

ให้สังเกตว่าคำถามข้างบนนี้จะมีค่าที่อยู่ในวงเล็บ หรือ Object ซึ่งใช้อักษรตัวใหญ่หน้าหน้า ได้แก่ Who, Father และ Son และโปรล็อกได้ทำการค้นหาคำตอบของค่าเหล่านี้ให้เราโดยการเทียบกับประโยคหรือฐานข้อมูลที่มีอยู่ในโปรแกรม ค่าที่ขึ้นต้นด้วยอักษรตัวใหญ่เหล่านี้ สามารถใช้แทนค่าใดๆก็ได้ที่สามารถจะเข้าคู่กับคำถามได้พอดี และเราเรียกว่าตัวแปร (Variable) การมีตัวแปรที่สามารถแทนค่าอื่นๆได้เช่นนี้ ทำให้โปรล็อกมีความยืดหยุ่นในการใช้งานได้ดีขึ้น

สำหรับกฎในโปรล็อกก็เป็นส่วนที่มีความสำคัญเท่ากับหลักการของข้อเท็จจริงที่ได้กล่าวมา เราจะกล่าวโดยสรุปในหลักการเบื้องต้นได้ว่าการเขียนโปรแกรมโปรล็อกจะประกอบด้วย

1. การบอกกล่าวถึงข้อเท็จจริงเกี่ยวกับวัตถุและความสัมพันธ์ระหว่างวัตถุ
2. การกำหนดกฎที่เกี่ยวข้องกับวัตถุและความสัมพันธ์ระหว่างวัตถุ
3. การตั้งเป่าประสงค์เกี่ยวกับวัตถุและความสัมพันธ์ระหว่างวัตถุ

2.3 ข้อเท็จจริง (Fact)

เนื่องจากโปรล็อกเป็นภาษาที่ใช้ตรรกวิทยาในการตัดสินใจ ฉะนั้นส่วนใหญ่ของโปรแกรมโปรล็อกจึงเป็นการรวบรวมฐานความรู้ (Knowledge Base) เกี่ยวกับเรื่องใดเรื่องหนึ่งโดยเฉพาะ ฐานความรู้นี้อาจจะเรียกได้ว่าเป็นฐานข้อมูล (Database) และในโปรล็อก เราจะเขียนฐานข้อมูลในรูปแบบของข้อเท็จจริง และกฎ (Facts and Rules) โปรล็อกให้เราสามารถบรรยายข้อเท็จจริงที่แสดงเป็นสัญลักษณ์ของความสัมพันธ์ได้ ตัวอย่างเช่น ถ้าเราต้องการจะบรรยายข้อเท็จจริงที่ว่า วิทูย์ไม่ดัง หรือ

The radio is dead.

เราจะสามารถแสดงในรูปแบบของโปรล็อกได้เป็น

is(radio, dead).

รูปแบบที่เราแสดงเป็นข้อเท็จจริงในภาษาโปรล็อกข้างบนนี้เรียกว่า อนุประโยค (clause) ให้สังเกตว่าตอนท้ายของอนุประโยคจะมีเครื่องหมายหักภาค . อยู่ด้วยเสมอ ในตัวอย่างนี้ is มีชื่อเรียกว่า Relation หรือความสัมพันธ์ ส่วน radio และ dead เรียกว่า วัตถุ (objects) ฉะนั้นจะเห็นได้ว่าคำว่าวัตถุไม่จำเป็นที่จะต้องเป็นวัตถุตามที่เคยเข้าใจกัน แต่จะเป็นคำนาม สรรพนาม อากาณาม กริยาหรือคำวิเศษณ์ก็ได้ วัตถุทั้งสองนี้จะขึ้นต้นด้วยตัวอักษรตัวเล็ก และมีชื่อเรียกว่า วัตถุประเภทสัญลักษณ์ (Symbol object) นอกจากต้องขึ้นต้นด้วยตัวอักษรตัวเล็กแล้ววัตถุประเภทสัญลักษณ์จะตามด้วยอักษรตัวใหญ่หรือตัวเลขก็ได้ แต่จะต้องมีจำนวนรวมกันไม่เกิน 250 ตัว และจะต้องไม่เว้นช่องว่างเอาไว้ ตัวอย่างต่อไปนี้แสดงถึงการเขียนข้อเท็จจริง และวัตถุประเภทสัญลักษณ์ที่ถูกต้อง

ภาษาอังกฤษ : The dog is white.
 Manop has a computer.
 Varunya likes chocolate.

ภาษาโปรล็อก :
Facts Relation Object
is(dog, white). is dog , white
has_a(manop, computer) has_a manop, computer
likes(varunya, chocolate).likes varunya, chocolate

คำบรรยายทั้งหมด ซึ่งไม่รวมถึงเครื่องหมายหักภาคด้วย มีชื่อเรียกว่า เพรดดิเคท(Predicate) และคำที่อยู่
ในวงเล็บของเพรดดิเคทเรียกว่า Argument ดังนั้นเพรดดิเคทของตัวอย่างข้างบนนี้ คือ

is(dog, white)
has_a(manop, computer)
likes(verunya, chocolate)

2.4 ตัวแปรในโปรล็อก

ตามที่ได้กล่าวมาแล้วว่า ตัวแปรในโปรล็อกจะต้องขึ้นต้นด้วยอักษรตัวใหญ่เสมอ และมีความยาวรวมกันไม่เกิน 250 อักขระ ตัวอักขระที่ต่อจากอักษรตัวแรกอาจจะเป็นตัวอักษรตัวใหญ่ ตัวเล็ก หรือตัวเลขก็ได้ ตัวอย่างเช่น

Goal : career(Profession, self_confidence)

โปรล็อกจะให้คำตอบว่า

Profession = air_hostess

Profession = engineer

2 Solutions

นั่นคือ โปรล็อกได้บอกเราว่าสำหรับผู้ที่มีความเชื่อมั่นในตนเองนั้น อาจจะมีอาชีพเป็นพนักงานต้อนรับบนเครื่องบิน หรือวิศวกรก็ได้

ให้สังเกตว่าคำว่า Profession นี้เรานำหน้าด้วยอักษรตัวใหญ่ จึงเป็นคนละคำกับ profession ที่อยู่ในโดเมนในการทำงานเดียวกันถ้าเราใช้ X แทน Profession เราก็จะได้คำตอบว่า

X = air hostess

X = engineer

2 Solutions

ในทางกลับกัน เราอาจจะถามหรือตั้งเป้าประสงค์ว่า ถ้าต้องการจะมีอาชีพเป็นแพทย์บุคคลนั้นควรจะมีคุณลักษณะอย่างไร เราจะได้คำตอบจากโปรแกรมดังนี้

Goal : career (doctor, X)

X = good_health

X = determination

X = devotion

3 Solutions

2.5 ตัวแปรที่ไม่ต้องการระบุตัวแปร

ในบางครั้งเราต้องการที่จะให้โปรแกรมไม่สนใจวัตถุบางวัตถุ (หรือบาง argument) เพื่อประโยชน์ในการควบคุมการทำงานของโปรแกรม ตัวแปรที่ไม่ระบุ จะช่วยให้เราบรรลุ ถึงวัตถุประสงค์นี้ได้ตัวแปรนี้ใช้แทนได้ด้วยเส้นใต้ (_) ดังตัวอย่าง

Goal : career (_ , self_confidence)

เราจะได้คำตอบว่า True ที่เป็นเช่นนี้ก็เพราะว่า ในวัตถุช่องแรกไม่มีตัวแปรใด ๆ (ความจริงมีตัวแปรที่ไม่ระบุ) ดังนั้นโปรแกรมจึงจับคู่เฉพาะวัตถุในช่องสุดท้ายเท่านั้น ดังนั้นการใช้ตัวแปรที่ไม่ระบุ เราจะได้คำตอบแต่เพียงว่า “เป็นจริง” หรือ “เป็นเท็จ” เท่านั้น

2.6 กฎ (Rule)

ในการทำให้โปรแกรมมีประสิทธิภาพดีขึ้น เราจะต้องเพิ่มกฎ (Rules) เข้าไปในฐานข้อมูล เพื่อให้โปรแกรมสามารถใช้เป็นกลไกในการตัดสินใจโดยอาศัยข้อเท็จจริงที่มีอยู่ได้ดีขึ้น กฎก็คือประโยคที่บ่งบอกให้ทราบว่าข้อเท็จจริงข้อหนึ่งจะเป็นจริงหรือไม่จะขึ้นอยู่กับข้อเท็จจริงอื่นๆที่ตามมา ตัวอย่างเช่น เราจะบอกว่าถ้าท่านมีสุขภาพแข็งแรง มีความเชื่อมั่นในตนเอง และมีลักษณะดีแล้ว ท่านก็อาจจะเป็นพนักงานต้อนรับบนเครื่องบินได้

ในรูปแบบของภาษาอังกฤษเราจะเขียนได้เป็น :

IF you have good health

AND you have self_confidence.

AND you have good appearance.

THEN you probably can be an air hostess.

จากประโยคข้างบนนี้ เราจะพบว่าข้อเท็จจริงที่ว่า ท่านจะเป็นพนักงานต้อนรับบนเครื่องบินได้หรือไม่ขึ้นอยู่กับข้อเท็จจริงอื่น ๆ เกี่ยวกับตัวท่านเอง ในโปรแกรมเราเรียกอนุประโยคเช่นนี้เรียกว่ากฎ ซึ่งสามารถเขียนเป็นโปรแกรมได้ดังนี้

career (air_hostess) if

qualification (good_health) and
 qualification (self_confidence) and
 qualification (good_appearance).

ให้สังเกตรูปแบบทั่วไปของการเขียนกฎในโปรล็อก นั่นคือ ผลสรุปจะขึ้นมาก่อน แล้วตามด้วยคำว่า if ต่อจากนั้นจึงเป็นรายการเงื่อนไขที่ผลสรุปนี้ต้องใช้อ้างอิง โดยที่แต่ละรายการของเงื่อนไขจะถูกเชื่อมต่อกันด้วยคำว่า and และเมื่อหมดเงื่อนไขทุกรายการแล้วก็จะมีเครื่องหมายหักภาค . เหมือนเช่นเคย ส่วนสรุปนี้เราเรียกได้ว่าเป็นเป้าหมายหลัก (Goal) และเงื่อนไขแต่ละเงื่อนไขก็ถือว่าเป็นเป้าหมายรอง (Sub Goal) เป้าประสงค์หลักมีชื่อเรียกว่า Head หรือส่วนหัว เงื่อนไขทุกเงื่อนไขรวมกันมีชื่อเรียกว่า Antecedent หรือส่วนหาง และเงื่อนไขแต่ละเงื่อนไขเรียกว่า Premise ฉะนั้นในที่นี้ส่วนหางจะประกอบด้วยสาย Premise ดังนั้นเป้าหมายหลักจะเป็นจริงได้ก็ต่อเมื่อเงื่อนไข หรือเป้าหมายรองทุกส่วนเป็นจริง แต่ถ้าเป้าหมายรองใดไม่เป็นจริงแล้ว เป้าประสงค์หลักก็ไม่เป็นจริงด้วย

ให้สังเกตว่าเราเขียนเป้าหมายรองแยกออกทีละบรรทัด ทั้งนี้เพื่อให้อ่านได้ง่าย ในทางปฏิบัติเราอาจจะเขียนไว้ในบรรทัดเดียวกันก็ได้ เนื่องจากในการเขียนโปรแกรมเราจะต้องใช้คำว่า if และ and บ่อยครั้ง ดังนั้น โปรล็อกจึงให้เราสามารถใช้สัญลักษณ์แทนได้ดังนี้ คือ

if แทนได้ด้วยสัญลักษณ์ :-

and แทนได้ด้วยสัญลักษณ์ ,

or แทนได้ด้วยสัญลักษณ์ ;

ฉะนั้น ในตัวอย่างข้างบน เราจึงสามารถเขียนเป็นโปรล็อกได้ดังนี้

career (air_hostess) :-

qualification (good_health) .

qualification (self_confidence).

qualification (good_appearance).

2.7 ความจริงบางประการในการทำงานของโปรล็อก

ถึงแม้ว่าโปรล็อกจะไม่ใช้ภาษาคอมพิวเตอร์ประเภท Procedural Language แต่โปรล็อกก็จะต้องทำตามขั้นตอนทั่ว ๆ ไป ดังต่อไปนี้

1. อนุประโยคที่มีเพรดิเคทเดียวกัน จะต้องอยู่รวมกันในโปรแกรม โดยจะแทรกเพรดิเคทอื่นเข้ามา ระหว่างกลางไม่ได้ตัวอย่างเช่น เพรดิเคท career (Person, _) จะต้องเรียงอยู่ด้วยกันเป็นต้น ทั้งนี้รวมไปถึงข้อเท็จจริงที่มีเพรดิเคทเดียวกันด้วย
2. การทำงานของโปรล็อกเพื่อจับคู่หรือ Unification จะเริ่มจากด้านบนลงมาด้านล่าง และจากด้านซ้ายไปยังด้านขวา
3. หลังจากที่โปรล็อกพบคำตอบแรกแล้ว โปรล็อกยังคงพยายามที่จะหาคำตอบต่อไปอีกจนกว่าจะได้ค้นหาคอบทุกอนุประโยคที่เกี่ยวข้องกับเป้าหมาย

4. ถ้าเราสลับที่ของอนุประโยคในโปรแกรม โปรล็อกก็ยังคงจะให้คำตอบเช่นเดิมเสมอ แต่ประสิทธิภาพของการค้นหาคำตอบ จะขึ้นอยู่กับ การเรียงเรียงตำแหน่งของอนุประโยค ดังนั้นเราจึงควรให้กฎที่มีโอกาสเกิดขึ้นน้อยที่สุดไปอยู่สุดท้ายของกฎทั้งหมด

2.8 การย้อนกลับ (Backtracking)

เป็นวิธีการค้นหาคำตอบของโปรล็อก โดยโปรล็อกจะเคลื่อนไปข้างหน้าและย้อนกลับ เพื่อพยายามหาทุกทางที่จะพิสูจน์เป้าหมายให้ได้ จนกว่าจะหมดวิธีการค้นหา ลักษณะการทำงานแบบนี้ของโปรล็อกมักจะพบอยู่ในโปรแกรมเสมอ ให้ลองพิจารณาจากตัวอย่างต่อไปนี้

บุคคลในแต่ละอาชีพควรมีคุณลักษณะอย่างไร จึงจะเหมาะสมกับอาชีพนั้น ในที่นี้ให้พิจารณา 3 อาชีพก่อนดังนี้

Air Hostess	:	good health	(สุขภาพแข็งแรง)
		self-confidence	(มีความเชื่อมั่นในตนเอง)
		good appearance	(มีลักษณะและหน้าตาดี)
Engineer	:	good at number	(เก่งคำนวณ)
		self-confidence	(มีความเชื่อมั่นในตนเอง)
		good common sense	(มีสามัญสำนึกดี)
Doctor	:	good health	(สุขภาพแข็งแรง)
		determination	(มีความตั้งใจแน่วแน่)
		devotion	(มีความเสียสละ)

สมมติว่าบุคคลหนึ่ง มีคุณสมบัติ 2 อย่าง และเราอยากจะทราบว่าบุคคลนั้นเหมาะสมกับอาชีพอะไร โดยการถามทั้งสองเงื่อนไขพร้อมกัน ในโปรล็อกก็อนุญาตให้เราดำเนินการได้โดยตั้งเป้าหมายที่ทั้ง 2 เงื่อนไข เช่นถ้าเราต้องการให้มีคุณสมบัติว่า มีสุขภาพแข็งแรงและมีความตั้งใจแน่วแน่ เราสามารถเขียนเป็นเป้าหมายได้ดังนี้

Goal : career(X, good_health)and career(X, determination)

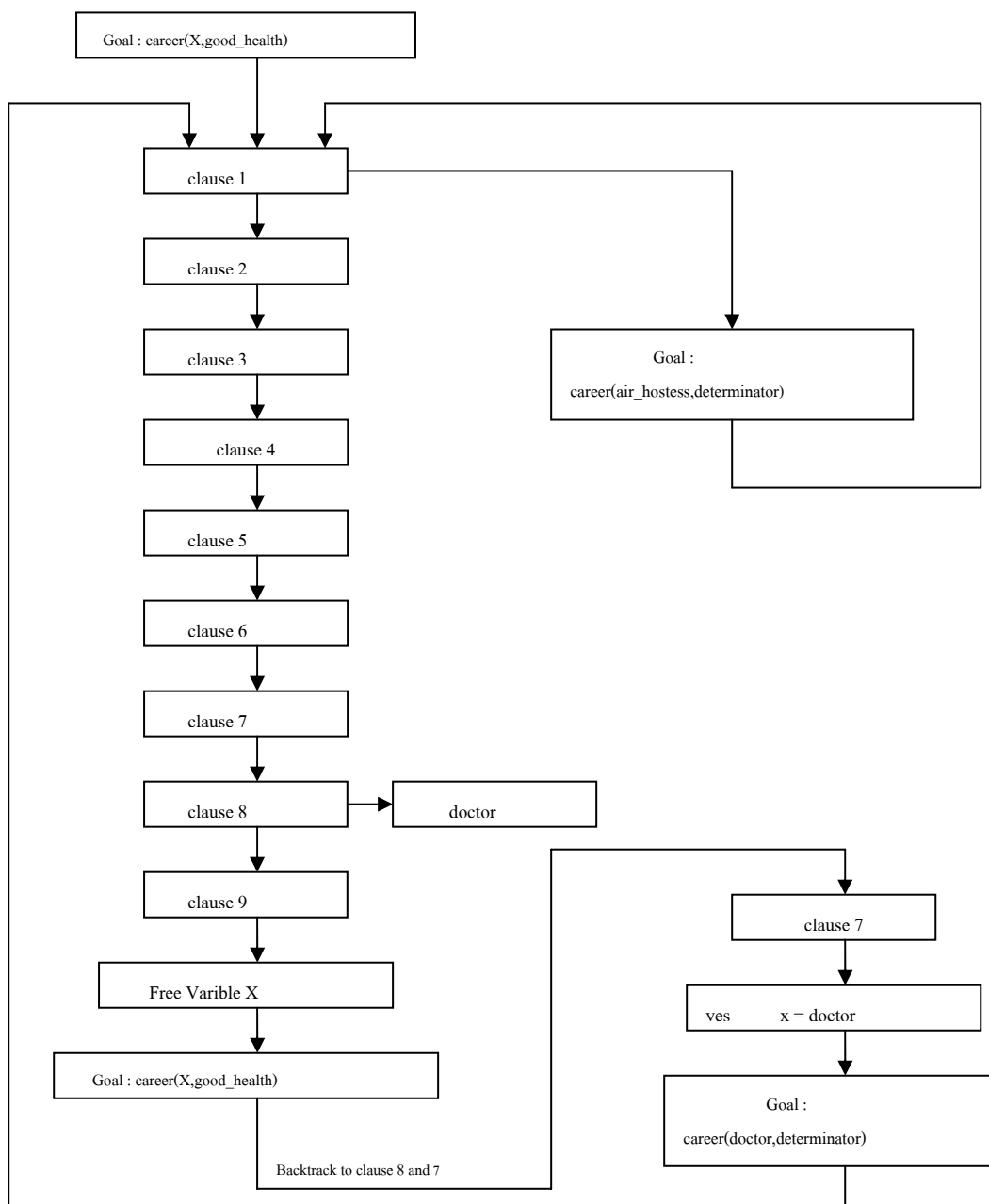
โปรล็อกจะให้คำตอบว่า

X = doctor

การทำงานของโปรล็อกในเป้าหมายนี้ แสดงให้เห็นถึงคุณสมบัติหรือวิธีการหาคำตอบที่สำคัญของ โปรล็อก เรียกว่า การย้อนกลับ (Backtracking) ซึ่งสามารถอธิบายโดยพิจารณารูปที่ 2-1 ประกอบ

โปรล็อกเริ่มต้นหาคำตอบโดยการพิจารณาเป้าหมายแรกคือ career(X, good_health) และเริ่มเทียบกับอนุประโยคที่ 1 ซึ่งปรากฏว่าสามารถจับคู่ได้พอดี ดังนั้นโปรล็อกจึงกำหนดค่าให้ตัวแปร Profession = air_hostess ก่อน จากนั้นโปรล็อกก็จะไปพิจารณาเป้าหมายที่ 2 ซึ่งจะกลายเป็น career(air_hostess, determination) โปรล็อกจะพยายามพิสูจน์ความจริงนี้โดยเริ่มตั้งแต่อนุประโยคที่ 1 ใหม่ แล้วเลื่อนลงมาเรื่อย ๆ จนถึงอนุประโยคที่ 9 ซึ่งเป็นอนุประโยคสุดท้าย โปรล็อกก็ยังไม่สามารถจับคู่กับอนุประโยคใดได้ ดังนั้นจึงถือว่าการค้นหาในครั้งนี้ fail หรือไม่ประสบผลสำเร็จ เมื่อเป็นเช่นนี้ โปรล็อกจะปล่อยตัวแปร Profession ให้เป็นอิสระจาก air_hostess แล้วไปพิจารณา

เป้าหมายแรกใหม่พร้อมกับพิจารณาย้อนกลับจากอนุประโยคที่ 9 ไป 8 และ 7 ซึ่งโปรล็อกจะพบว่าสามารถจับคู่เป้าหมายแรกคือ $\text{career}(X, \text{good_health})$ กับ $\text{career}(\text{doctor}, \text{good_health})$ ได้พอดี ฉะนั้นโปรล็อกจึงกำหนดให้ $X = \text{doctor}$ จากนั้นโปรล็อกก็จะไปพิจารณาเป็นเป้าหมายที่ 2 ใหม่ ซึ่งขณะนี้จะกลายเป็น $\text{career}(\text{doctor}, \text{determination})$ โดยย้อนกลับไปเริ่มตรวจสอบตั้งแต่อนุประโยคที่ 1 ว่าจะมีอนุประโยคใดจับคู่ได้พอดีหรือไม่ จนกระทั่งถึงอนุประโยคที่ 8 ก็พบอนุประโยคที่จับคู่ได้พอดี เมื่อโปรล็อกสามารถจับคู่หรือพิสูจน์เป้าหมายทั้งสองนี้ได้ครบถ้วน ก็จะสรุปคำตอบออกมาว่า $X = \text{doctor}$



รูปที่ 2-1 ตัวอย่างแสดงการย้อนกลับ (Backtracking)

2.9 เพรดดิเคทเฟล (Fail)

เฟล (fail) เป็น Built-in Predicate ที่มีอยู่ในโปรล็อก เมื่อโปรล็อกพบเฟลในอนุประโยคใดก็จะมีผลให้อนุประโยคนั้นเป็นเท็จทันทีซึ่งจะทำให้โปรล็อกต้องย้อนกลับหรือBacktrackingไปยังอนุประโยคที่เหมาะสมต่อไปจะเห็นได้ว่าเฟลเป็นคำสั่งหรือเพรดดิเคทประเภทที่ใช้ควบคุมการทำงานของโปรล็อกเพื่อให้ประสบความสำเร็จโดยการบังคับหรือเราอาจจะพูดได้ว่าเป็นการใช้ความไม่สำเร็จช่วยให้เกิดความสำเร็จ ให้เราสังเกตโครงสร้างทั่วไปของโปรแกรมซึ่งใช้ fail ว่ากฎที่มีเพรดดิเคท fail อยู่จะไม่ประสบผลสำเร็จในการพิสูจน์เป้าหมายประสงค์ทุกครั้ง และเพื่อให้โปรแกรมสามารถทำงานได้อย่างเหมาะสม เราจะต้องตามกฎนี้ด้วยอนุประโยคที่เป็นจริงเสมอ อนุประโยคเหล่านี้เรียกว่า terminating Condition นั่นคือกฎแรกจะทำงานทั้งหมดตามที่เรากำลังต้องการ ส่วนอนุประโยคที่เป็นจริงเสมอซึ่งอยู่ถัดมาจะหยุดการถอยย้อนกลับของโปรล็อก

เราสามารถสรุปการทำงานของโปรล็อกได้ว่า ในกรณีที่มีกฎมากกว่าหนึ่งกฎที่มีส่วนหัวเหมือนกัน และต่างก็จบลงด้วยเพรดดิเคท fail นั้น หลังจากที่โปรล็อกพยายามพิสูจน์กฎแรกจนครบทุกเพรดดิเคท หรือเงื่อนไขแล้วโปรล็อกก็จะเลื่อนลงมาที่กฎต่อไปซึ่งมีส่วนหัวเหมือนกัน และพยายามพิสูจน์กฎเช่นเดียวกันจนกระทั่งโปรล็อกเลื่อนลงมาถึงอนุประโยคหรือข้อเท็จจริง ซึ่งไม่มีเพรดดิเคท fail อยู่เป้าหมายจึงจะประสบผลสำเร็จ

2.10 เพรดดิเคทคัท (Cut)

คัท (Cut) เป็น built-in Predicate ที่มีความสำคัญมากที่สุดเพรดดิเคทหนึ่ง และเป็นเพรดดิเคทที่มีการใช้งานที่ค่อนข้างจะซับซ้อน จนบางครั้งเราอาจจะติดตามการทำงานของโปรล็อกได้ยาก การจะเข้าใจเพรดดิเคทคัทได้ดี ผู้อ่านจะต้องเข้าใจเกี่ยวกับการถอยย้อนกลับหรือ Backtracking ของโปรล็อกเป็นอย่างดีด้วย

วัตถุประสงค์หลักของคัทก็คือการป้องกันมิให้มีการถอยย้อนกลับหลังจากที่โปรล็อกได้พิสูจน์บางเงื่อนไขหรือเพรดดิเคทได้แล้วเพรดดิเคทคัทนี้ใช้แทนด้วยสัญลักษณ์หรือเครื่องหมาย ! เมื่อโปรล็อกเลื่อนมาจนถึงคัทแล้วโปรล็อกจะผ่านคัทไปได้เสมอ นั่นคือ เพรดดิเคทคัท จะให้ผลที่เป็นจริงเสมอ แต่เมื่อผ่านคัทไปแล้วโปรล็อกจะผ่านคัทมาใหม่ไม่ได้ และโปรล็อกจะต้องข้ามไปยังอนุประโยคอื่นที่เหมาะสมต่อไป แต่ถ้าคัทเป็นเพรดดิเคทสุดท้าย และไม่มีกฎใด ๆ ที่โปรล็อกจะต้องพยายามพิสูจน์ต่อไปอีกแล้ว โปรล็อกก็จะรับผลตามที่ได้ในขณะนั้นไม่ว่าผลนั้นจะเป็นจริงหรือเท็จ

การใช้คัทเป็นการบังคับโปรล็อกให้ยอมรับผลที่ได้ในขณะนั้นโดยไม่ต้องกลับไปตรวจสอบอนุประโยคอื่นๆอีก ดังนั้นการใช้คัทในที่ที่เหมาะสม จึงช่วยให้โปรแกรมทำงานได้ตามวัตถุประสงค์ เพรดดิเคทเฟลและคัทที่กล่าวมานี้ต่างก็มีลักษณะการบังคับการทำงานของโปรล็อก ซึ่งเข้าลักษณะของ Procedural Programming อันขัดต่อหลักการของโปรล็อกแต่ก็จำเป็นที่จะต้องมียูเพื่อไม่ให้โปรแกรมสามารถทำงานได้ในบางกรณีดังเช่นตัวอย่างเหล่านี้ และยังจัดได้ว่าทั้งเพรดดิเคทเฟลและคัทต่างก็มีความสำคัญมากในโปรล็อกอีกด้วย

2.11 การตัดสินใจในการใช้คัท

วัตถุประสงค์พื้นฐานของเพรดดิเคทคัท ก็คือช่วยให้โปรล็อกสามารถตัดทอนทิศทางการค้นหาเพื่อพิสูจน์ความจริงบางทิศทางออกไป ซึ่งอาจจะช่วยให้โปรแกรมทำงานได้เร็วขึ้น ซึ่งเราสามารถที่จะทำความเข้าใจโดยสรุปจากกฎต่อไปนี้คือ

run :-

predicate1, predicate2 ,!,

predicate3 , predicate4 ,!,

หลักการทำงานของโปรล็อก คือ โปรล็อกจะพยายามพิสูจน์โดยการเลื่อนย้อนกลับไประหว่าง predicate1 และ predicate2 ตามความจำเป็นจนกระทั่งทั้งสองเพรดิเคทเป็นจริง เมื่อทั้งสองเพรดิเคทได้รับการพิสูจน์ไปแล้ว โปรล็อกก็จะมาถึงคัทเนื่องจากคัทจะเป็นจริงเสมอ ดังนั้นโปรล็อกจึงเลื่อนต่อมายัง predicate3 หลังจากที่โปรล็อกเลือกผ่านคัทมาแล้ว โปรล็อกจะไม่สามารถถอยย้อนกลับผ่านคัทไปได้อีก ดังนั้นโปรล็อกจึงต้องพิสูจน์กลับไปกลับมาะหว่าง predicate3 และ predicate4 ถ้า predicate3 หรือ predicate4 ไม่เป็นจริง run ก็จะไม่เป็นจริงด้วย แต่ก่อนที่โปรล็อกจะผ่านคัทมาค่าต่าง ๆ ของตัวแปรจะถูกกำหนดโดย Predicate1 และ predicate2 แล้วฉะนั้นโปรล็อกจะให้ค่าเหล่านี้โดยอัตโนมัติ การใช้คัทนี้นอกจากจะใช้ร่วมกับเฟลด์ังตัวอย่างที่แสดงมาแล้วยัง มักจะใช้ร่วมกับการอ้างตนเอง(Recursion)อีกด้วย ในขณะนี้เราอาจจะสรุปได้ว่า คัทอาจจะใช้สำหรับวัตถุประสงค์ใดวัตถุประสงค์หนึ่งต่อไปนี้

1. เพื่อหยุดการค้นหาคำตอบเพิ่มเติมหลังจากที่กฎใดกฎหนึ่งที่กำหนดไว้ได้รับการทดสอบแล้ว
2. เพื่อหยุดการค้นหาคำตอบเพิ่มเติมหลังจากที่กฎใดกฎหนึ่งที่กำหนดไว้ถูกบังคับให้เป็นเท็จโดยเพรดิเคทเฟล
3. สำหรับจัดทิศทางค้นหาในฐานข้อมูลบางทิศทาง เพื่อช่วยให้การทำงานของโปรล็อกเร็วขึ้น